Computer Vision Winter 2026 - Exercise 1

Machine Learning for Computer Vision - TU Dresden

October 26 - 30, 2026

OpenCV

In the exercise, we will use the OpenCV library. To install OpenCV, visit the official OpenCV website: `https://opencv.org/get-started/`, select your operating system and C++ as the programming language, and then follow the instructions under "Run this Command".

To build an OpenCV project we recommend you use CMake `https://cmake.org/cmake/help/book/mastering-cmake/chapter/Getting%20Started.html`. The CMakeLists file required to build the OpenCV project used in this exercise is provided through OPAL.

Note for Windows users: OpenCV only provides prebuilt binaries for the latest versions of the Visual Studio toolset. Therefore if you want to use a different compiler you will need to build the OpenCV library from source.
Furthermore the Windows .exe package does not perform an automatic installation or environment setup—it merely extracts the prebuilt binaries. Therefore, you will need to do the following steps manually:

- Add binaries i.e. `...\opencv\build\x64\vc16\bin` to system PATH
- Provide CMake with the location of OpenCV. This can be done via the command line interface of CMake with: `-DOpenCV_DIR="...\opencv\build\x64\vc16\lib"` or via the GUI by adding a new entry with name: `OpenCV_DIR` and the path to OpenCV as the value.

Point Operators

In this task we will implement a point operator, namely a simple color inversion. Recall the definition of a point operator φ for a digital image f, color space C and function χ as seen in the lecture:

$$\varphi(f)(x, y) = \chi(f(x, y), x, y)$$

For this task we choose to use grayscale images, i.e. $C = \{0, ..., 255\}$. The function χ defining a color inversion is as follows:

$$\chi : f(x, y) \mapsto 255 - f(x, y)$$

1. Read the image `fruits_grayscale.jpg` as a grayscale image using the `imread` function of OpenCV. To ensure the image has been loaded correctly you can display the image using the `imshow` function of OpenCV.
2. Apply the color inversion point operator to the `fruits_grayscale.jpg` image by looping over all pixels of the image and applying χ to each pixel. You can access a single pixel by use of the `at` member function of the OpenCV `cv::Mat` class. Display your result.


(a) Input Image


(b) Inverted Image

Histogram equilibration

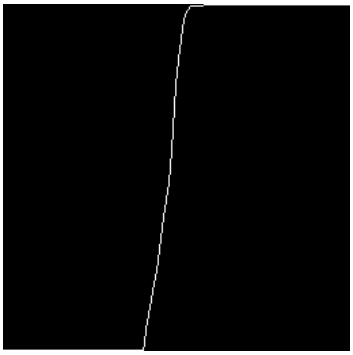
In this task we will implement histogram-equilibration/self-equilibration of a grayscale image as seen in the lecture.

3. To implement histogram-equilibration we will follow the steps presented in the lecture:

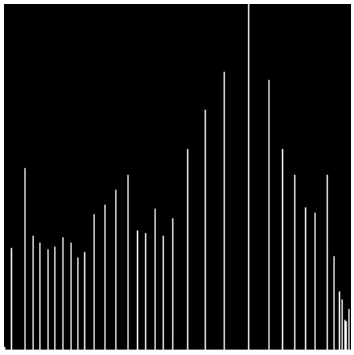
   3.1 Calculate the histogram of the `fruits_low_contrast.jpg` image and display the result.

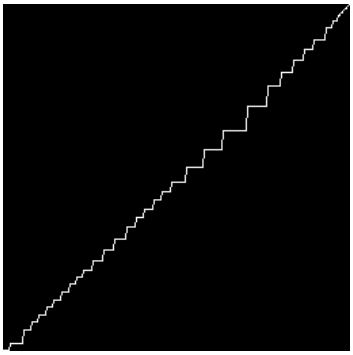
   3.2 Calculate the cumulative distribution of colors(CDC) from the histogram and display the result.


(a) Histogram


(b) Cumulative Distribution of Colors

   3.3 Using the cumulative distribution of colors calculate the self-equilibration of the `fruits_low_contrast.jpg` image. Display both the histogram and cumulative distribution of colors of the equalized image.


(a) Equalized Histogram


(b) Equalized Cumulative Distribution of Colors

4. To validate our results we can use the histogram-equilibration implemented in OpenCV (`equalizeHist`) applied to the `fruits_low_contrast.jpg` image.



(a) Input Image (b) Equalized Image